

```
# Import required libraries
import pandas as pd
from sklearn.preprocessing import LabelEncoder
```

```
# Create dummy hospital dataset

data = {
    'Gender': ['Male', 'Female', 'Female', 'Male', 'Female'],
    'AppointmentType': ['General', 'Dental', 'Cardiology', 'General', 'Dental'],
    'City': ['Kochi', 'Kottayam', 'Calicut', 'Kochi', 'Calicut']
}

# Convert dictionary into DataFrame
df = pd.DataFrame(data)

# Display dataset
print(df)
```

	Gender	AppointmentType	City
0	Male	General	Kochi
1	Female	Dental	Kottayam
2	Female	Cardiology	Calicut
3	Male	General	Kochi
4	Female	Dental	Calicut

```
# Display dataset information
print(df.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Gender           5 non-null      object
1   AppointmentType  5 non-null      object
2   City             5 non-null      object
dtypes: object(3)
memory usage: 252.0+ bytes
None
```

```
# Statistical summary
print(df.describe())
```

	Gender	AppointmentType	City
count	5	5	5
unique	2	3	3
top	Female	General	Kochi
freq	3	2	2

```
# Create copy of dataset
label_encoded_df = df.copy()

# Initialize LabelEncoder
le = LabelEncoder()

# Apply Label Encoding to each column
label_encoded_df['Gender'] = le.fit_transform(label_encoded_df['Gender'])

label_encoded_df['AppointmentType'] = le.fit_transform(label_encoded_df['AppointmentType'])

label_encoded_df['City'] = le.fit_transform(label_encoded_df['City'])

# Display encoded dataset
print(label_encoded_df)
```

	Gender	AppointmentType	City
0	1	2	1
1	0	1	2
2	0	0	0
3	1	2	1
4	0	1	0

✓ Observation of Label Encoding

Label Encoding converts categorical values into numerical labels.

Example:

- Male $\rightarrow$ 1
- Female $\rightarrow$ 0

Advantages:

- Simple and memory efficient
- Good for ordinal data

Disadvantages:

- Machine learning models may assume numerical order
- Not suitable for nominal categorical data

```
# Apply One-Hot Encoding
one_hot_encoded_df = pd.get_dummies(df)
```

```
# Display encoded dataset
print(one_hot_encoded_df)
```

```
Gender_Female  Gender_Male  AppointmentType_Cardiology \
0             False         True                      False
1             True         False                      False
2             True         False                      True
3             False        True                      False
4             True         False                      False

AppointmentType_Dental  AppointmentType_General  City_Calicut  City_Kochi \
0                False                True          False         True
1                True                False          False         False
2                False                False          True         False
3                False                True          False         True
4                True                False          True         False

City_Kottayam
0          False
1           True
2          False
3          False
4          False
```

✓ Observation of One-Hot Encoding

One-Hot Encoding creates separate binary columns for each category.

Example:

- Gender_Male
- Gender_Female

Advantages:

- No false numerical relationship
- Better for nominal categorical data

Disadvantages:

- Increases number of columns
- Uses more memory

```
# Compare shapes

print("Original Dataset Shape:", df.shape)

print("Label Encoded Shape:", label_encoded_df.shape)

print("One-Hot Encoded Shape:", one_hot_encoded_df.shape)
```

```
Original Dataset Shape: (5, 3)
Label Encoded Shape: (5, 3)
One-Hot Encoded Shape: (5, 8)
```

Comparative Analysis

Label Encoding

- Converts categories into numeric labels
- Dataset size remains same
- Suitable for ordinal data
- Can create misleading relationships

One-Hot Encoding

- Creates binary columns
- Dataset expands in size
- Suitable for nominal data
- Avoids false ordering problem

Conclusion

One-Hot Encoding is generally preferred for nominal categorical variables, while Label Encoding is useful for ordinal categories.